

PHENIKAA UNIVERSITY
PHENIKAA SCHOOL OF COMPUTING



SUBJECT: SOFTWARE ARCHITECTURE
PROJECT FINAL REPORT: MOVIE TICKET BOOKING SYSTEM

Instructor : Vu Quang Dung
Group 8: Bui Minh Duc (23010513)
Pham Xuan Bach (23010118)
Nguyen Gia Bao (23010383)
Vu Duc Hieu (23010226)
Course class: CSE703110-1-2-25(N01)

Ha Noi, March 2026

Contents

1. EXECUTIVE SUMMARY	7
1.1 Goal:	7
1.2 Final Achievement:.....	7
2. PROJECT REQUIREMENTS & GOALS	7
2.1 Functional Requirements (Core Features).....	7
2.2 Quality Attributes (Targeted Goals).....	7
3. ARCHITECTURAL DESIGN & IMPLEMENTATION	8
3.1 Technical Stack.....	8
3.2 Implementation of Core Logic (Operational Phases).....	8
3.3 System Architecture Diagram	9
3.4 Deployment Diagram	10
4. Requirements Analysis & Use Case Specifications	10
4.1 Functional Requirements (FR)	10
4.1.1 User Management & Authentication	10
4.1.2 Movie Catalog Management	11
4.1.3 Room & Showtime Management	11
4.1.4 Seat & Booking Management.....	12
4.1.5 Payment	12
4.1.6 Notifications (Event-Driven).....	13
4.1.7 API Gateway & Access Control.....	13
4.2 Non-Functional Requirements (NFR)	13
4.3 Actors.....	14
4.4 Use Case Diagram	15
5. USE CASE SPECIFICATIONS.....	15
5.1 System & Authentication (UC-01 to UC-03).....	15
5.2 Movie & Showtime Management (Admin) (UC-04 to UC-09)	17
5.3 Browsing & Booking (User) (UC-10 to UC-15).....	23

5.4	Payment (UC-16).....	27
6.	DETAILED OPERATIONAL MECHANISMS, INTERACTIONS, AND DATA FLOW	28
6.1	C4 Model	28
6.2	Communication Architecture	33
6.3	Routing Mechanism and API Gateway Pattern.....	33
6.4	Detailed Workflows Analysis.....	35
6.5	Distributed Data Management.....	37
6.6	Fault Tolerance	37
7.	IMPLEMENTATION: CLIENT-SIDE LOGIC	38
7.1	Customer.....	38
7.2	Admin	44
7.2.1	Introduction.....	44
7.2.2	Authentication and Authorization	44
7.2.3	Admin Dashboard.....	46
7.2.4	Movie Management	48
7.2.5	Showtime Management.....	49
7.2.6	Booking (Ticket) Management	52
7.2.7	User Account Management.....	53
–	Updating User Role and Password	54
–	Gateway Routing	54
7.2.8	Summary of Admin Operations	54
8.	TESTING & VERIFICATION	55
8.1	Verification of Requirements (Labs Progress).....	55
8.2	Quality Attribute Testing.....	56
8.3	Test Summary	58
9.	CONCLUSION & REFLECTION	59
9.1	Summary of Lessons Learned	59
9.2	Future Improvements.....	59

Work Assignment Table

Member	Detailed Tasks	Completion Rate
Nguyen Gia Bao	1. Movie Service: Implemented CRUD operations for Movies and Showtimes; handled Cascade Delete logic. 2. Booking Service: Developed core booking logic, atomic transactions for seat locking, and OrderPlaced event publishing. 3. Database Design: Designed and optimized SQLite schemas for movies.db and booking.db. 4. Testing: Conducted Fault Tolerance and resilience testing.	100%
Pham Xuan Bach	1. Infrastructure: Created Dockerfile, docker-compose.yml, and configured the RabbitMQ network. 2. Notification Service: Implemented the Consumer worker to handle email sending and invoice generation events. 3. Payment Service: Developed the Payment REST API and INVOICE_PRINT event publishing.	100%
Bui Minh Duc	1. System Architecture: Designed the C4 Model and defined the Tech Stack. 2. Gateway Service: Developed the API Gateway (app.py), configured request routing and security headers. 3. User Service: Implemented Authentication (Login/Register), Token Verification (UUID), and User Management. 4. Documentation: Wrote the Executive Summary, System Use Cases, and consolidated the Final Report.	100%
Vu Duc Hieu	1. UI/UX Design: Built a responsive SPA frontend using Vanilla JS (ES6+), Bootstrap 5, and Vite; implemented component-based structure, state handling, and client-side routing. 2. Client Integration: Developed a reusable API client layer (myClient.js) handling request abstraction, error handling, and token-based authentication with the Gateway.	100%

	3. Admin Pages: Implemented admin dashboard modules (Movies, Showtimes, Analytics) with CRUD flows, form validation, and dynamic table rendering. 4. User Pages: Built the Homepage, Movie Details, Booking flow, and Profile pages.	
--	--	--

1. EXECUTIVE SUMMARY

1.1 Goal:

The primary objective of this project was to refactor a legacy Layered Monolith movie ticket booking application into a distributed Microservices Architecture.

Architectural Pattern: The system adopts a Microservices style utilizing Database-per-Service isolation and a Hybrid Communication Pattern (Synchronous REST APIs for reads and Asynchronous AMQP via RabbitMQ for writes).

1.2 Final Achievement:

The team successfully decomposed the application into independent containers (Gateway, Movie, Booking, Payment, Notification). The final system is orchestrated via Docker Compose, demonstrating high scalability, fault tolerance, and a clear separation of concerns, successfully addressing the limitations of the previous monolithic version.

2. PROJECT REQUIREMENTS & GOALS

2.1 Functional Requirements (Core Features)

- **Movie Management (CRUD):** Admin capabilities to Create, Read, Update, and Delete movies and showtimes.
- **Booking Transaction:** A complex workflow allowing users to select seats, lock them, and process payments.
- **Real-Time Messaging:** Event-driven notifications (Email/Tickets) and Invoice generation upon successful booking.
- **Search & Discovery:** High-performance movie searching for end-users.

2.2 Quality Attributes (Targeted Goals)

- **Scalability:** The ability to scale specific services (e.g., Booking Service) independently during high-traffic events.
- **Availability & Fault Tolerance:** The system must continue to accept bookings even if the Notification or Payment services are temporarily down (achieved via Message Queues).
- **Performance:**
 - *Read Performance:* Fast response times for movie listings using direct API calls.

- *Write Performance:* Non-blocking booking experience using asynchronous processing.

3. ARCHITECTURAL DESIGN & IMPLEMENTATION

3.1 Technical Stack

- Backend: Python 3.13 (Flask) for lightweight microservices.
- Frontend: Vanilla JavaScript (ES6+) and Bootstrap 5, bundled with Vite.
- Database: SQLite (Strict Database-per-Service implementation).
- Message Broker: RabbitMQ (Pika client) for event-driven communication.
- Infrastructure: Docker.

3.2 Implementation of Core Logic (Operational Phases)

Phase A: Infrastructure Bootstrap (Containerization)

- Orchestration: We use `docker-compose.yml` as the infrastructure blueprint. An internal Bridge Network is established to allow services to communicate via Hostnames (Service Discovery) rather than static IPs.
- Idempotent Initialization: Upon startup, each service checks for its specific database file (`movies.db`, `booking.db`). If absent, it automatically executes SQL schemas to create tables and seed initial data. This ensures strict data isolation with no cross-service foreign keys.

Phase B: Synchronous Operations (Read-Heavy Flows)

- API Gateway Pattern: The Flask Gateway (Port 5005) acts as the Single Entry Point. It intercepts Frontend requests, validates UUID-based Security Tokens stored in the database, and routes traffic to internal services (e.g., routing `/api/movies` to `movie-service:5001`).
- Direct Querying: For information retrieval, the Gateway forwards requests to the Movie Service, which queries its local DB and returns JSON directly. This minimizes latency for UI rendering.

Phase C: Asynchronous Operations (Write-Heavy Flows)

- Event-Driven Booking:
 1. Atomic Transaction: The Booking Service locks the seat in its local DB.
 2. Producer: Immediately upon success, it publishes an *OrderPlacedEvent* directly to the *ticket_events* queue.

3. Non-Blocking Response: The user receives a "Success" HTTP 201 response immediately, without waiting for email processing.

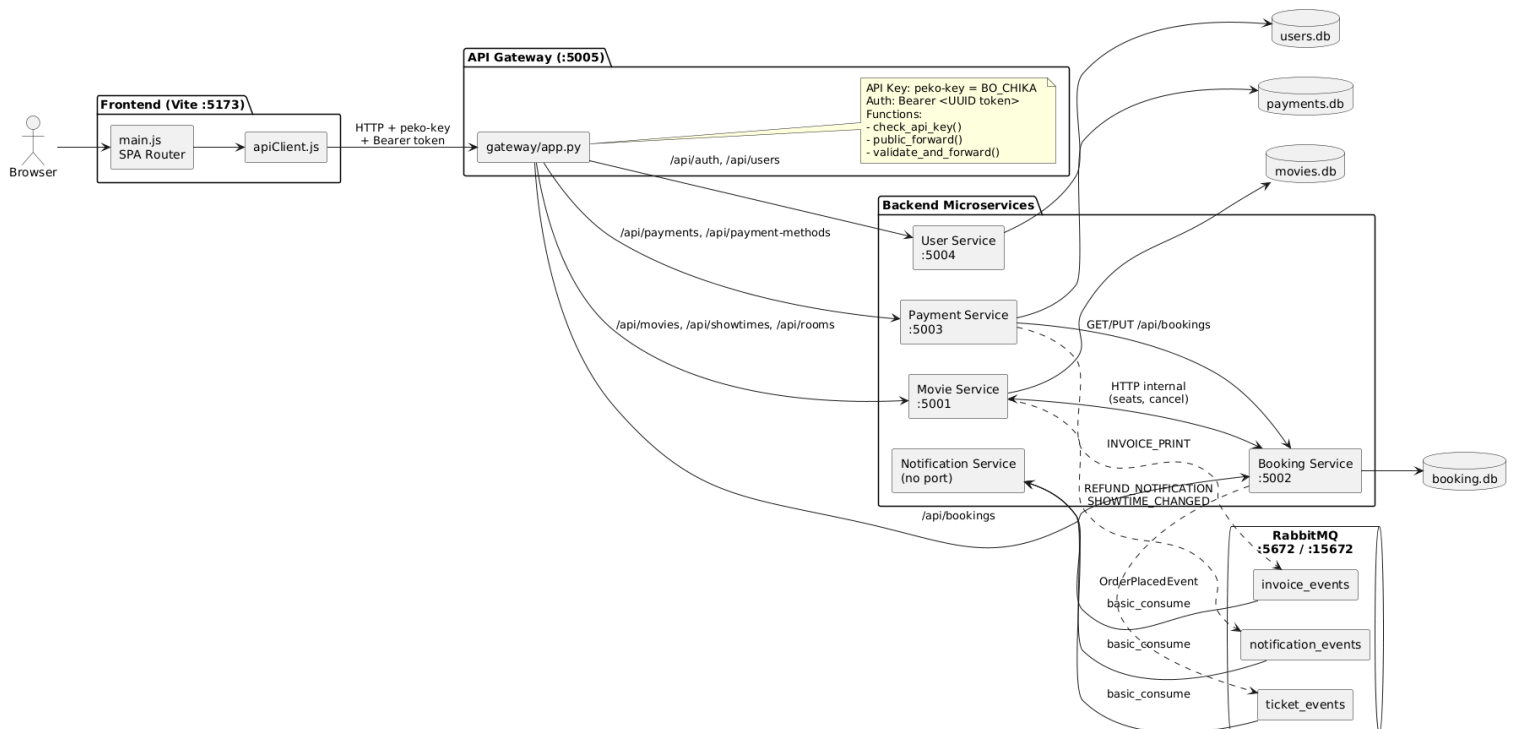
- **Consumer Workers:** The Notification and Payment services run as background workers. They consume messages from the queue to generate invoices and send emails via SMTP. If a worker crashes, the message remains in the queue (Persistence), ensuring no data loss.

Phase D: Technical Optimization

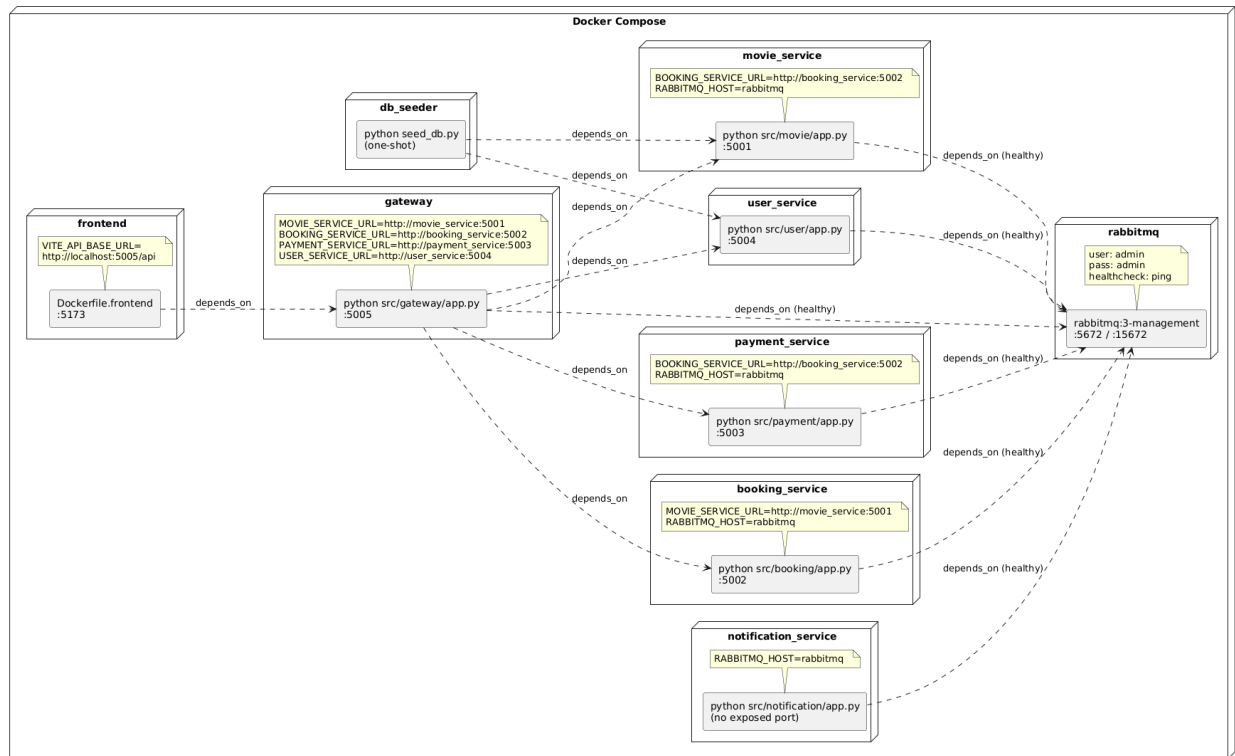
- **Frontend Debounce:** To prevent API overload, we implemented a custom useDebounce hook in React. This limits search requests by only firing after the user stops typing for 500ms, reducing Gateway load by approximately 80%.

Below is the rewritten content formatted cleanly so you can paste it directly into Microsoft Word (structured with headings, numbered sections, and tables).

3.3 System Architecture Diagram



3.4 Deployment Diagram



4. Requirements Analysis & Use Case Specifications

4.1 Functional Requirements (FR)

The following functional requirements are derived from the implemented system behavior across all microservices and the frontend application.

4.1.1 User Management & Authentication

ID	Requirement
FR-01	The system shall allow new users to register with email and password.
FR-02	The system shall authenticate users via email/password and issue a UUID session token.
FR-03	The system shall verify session tokens to authorize protected API requests.

ID	Requirement
FR-04	The system shall support two roles: customer and admin.
FR-05	The system shall allow authenticated users to view their profile (id, email, role).
FR-06	The system shall allow admins to list, view, and update users (role/password).

4.1.2 Movie Catalog Management

ID	Requirement
FR-07	The system shall allow anyone to browse and search movies by title or genre.
FR-08	The system shall allow anyone to view movie details (title, genre, duration, release date, description, poster).
FR-09	The system shall allow admins to create, update, and delete movies.
FR-10	The system shall allow admins to upload movie poster images.
FR-11	The system shall cascade-delete all showtimes and cancel/refund all related bookings when a movie is deleted.

4.1.3 Room & Showtime Management

ID	Requirement
FR-12	The system shall allow anyone to list screening rooms and view room details.
FR-13	The system shall allow admins to create showtimes (movie, room, start/end time, price).
FR-14	The system shall validate that showtime duration covers the movie's runtime.
FR-15	The system shall allow admins to update showtimes; price/room changes trigger refund and cancellation of affected bookings.
FR-16	The system shall allow admins to delete showtimes with automatic refund of affected bookings.

ID	Requirement
FR-17	The system shall notify affected customers when showtime times change.

4.1.4 Seat & Booking Management

ID	Requirement
FR-18	The system shall display an interactive visual seat map with three tiers: STANDARD, VIP, COUPLE.
FR-19	The system shall allow authenticated customers to book multiple seats in a single transaction (grouped booking).
FR-20	The system shall atomically lock seats to prevent double-booking.
FR-21	New bookings shall start with status PENDING_PAYMENT.
FR-22	The system shall allow customers to view their own bookings.
FR-23	The system shall allow admins to list all bookings.
FR-24	The system shall allow deletion/cancellation of a booking and release associated seats.
FR-25	Seats shall be lazily generated on first access based on room dimensions.

4.1.5 Payment

ID	Requirement
FR-26	The system shall allow users to save credit card payment methods (only last 4 digits stored).
FR-27	The system shall allow CRUD operations on saved payment methods with ownership verification.
FR-28	The system shall process payments by updating booking status to confirmed and publishing an invoice event.

4.1.6 Notifications (Event-Driven)

ID	Requirement
FR-29	The system shall send booking confirmation notifications.
FR-30	The system shall send invoice notifications when payment is processed.
FR-31	The system shall send refund notifications when bookings are cancelled.
FR-32	The system shall send schedule change notifications when showtime times are modified.

4.1.7 API Gateway & Access Control

ID	Requirement
FR-33	All external API requests must pass through a single API Gateway.
FR-34	Every request (except public assets) must include a valid API key.
FR-35	The gateway shall enforce role-based access control.
FR-36	The gateway shall inject authenticated user headers into forwarded requests.

4.2 Non-Functional Requirements (NFR)

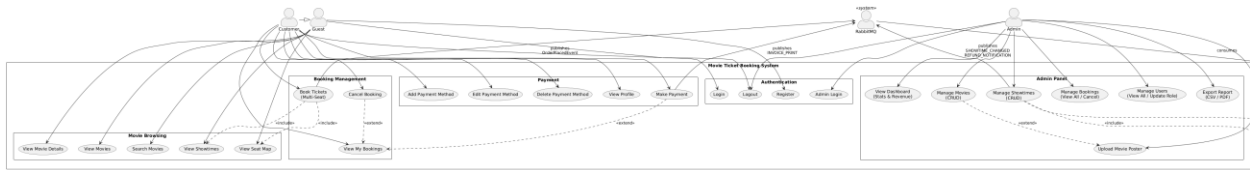
ID	Category	Requirement
NFR-01	Architecture	The system shall follow a microservices architecture.
NFR-02	Data Isolation	Each service shall own its own database.
NFR-03	Concurrency	Seat reservation shall use atomic operations to prevent double-booking.
NFR-04	Security	Passwords shall be securely hashed and never stored in plaintext.
NFR-05	Security	Credit card numbers shall be masked; only last 4 digits retained.

ID	Category	Requirement
NFR-06	Resilience	Notification failures shall not block booking or payment transactions.
NFR-07	Resilience	The Notification Service shall retry message broker connections indefinitely.
NFR-08	Messaging	Asynchronous communication shall use durable messaging queues.
NFR-09	Deployability	The system shall be deployable via Docker Compose.
NFR-10	Usability	The frontend shall be a Single Page Application with client-side routing.
NFR-11	Portability	Backend services shall use SQLite for zero-configuration setup.
NFR-12	Maintainability	Backend services shall follow layered architecture.

4.3 Actors

Actor	Description
Guest	Unauthenticated user who can browse movies and register/login.
Customer	Authenticated user who can book tickets and manage payments.
Admin	Authenticated user who can manage movies, showtimes, users, and bookings.
Movie Service	Provides movie and showtime data.
Booking Service	Manages seats and bookings.
Payment Service	Processes payments and invoice events.
Notification Service	Handles event-driven notifications.
Message Broker	Facilitates asynchronous communication.

4.4 Use Case Diagram



5. USE CASE SPECIFICATIONS

5.1 System & Authentication (UC-01 to UC-03)

UC-01: API Gateway Routing & Verification: Handles request routing, validates API Keys, and verifies User Tokens.

ID and Name:	UC-01: API Gateway Routing & Verification
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User, Admin
Description:	The Gateway acts as the single entry point (Port 5005). It intercepts all requests, validates the API Key, verifies User Tokens (if present), and routes the request to the appropriate microservice.
Trigger:	Any HTTP Request sent to http://localhost:5005
Preconditions:	PRE-1: Gateway Service is running on Port 5005. PRE-2: User Service is running to verify tokens.
Postconditions:	POS-1: Request is successfully forwarded to the downstream service. POS-2: User identity headers (X-User-Email, X-User-Role) are injected.
Normal flow:	1. Client sends a request with Header peko-key. 2. Gateway validates peko-key. 3. Gateway checks for Authorization: Bearer <token> header. 4. Gateway calls User Service to check token existence in <i>tokens</i> table. 5. User Service returns user profile and role. 6. Gateway injects user info into headers. 7. Gateway forwards request to the target service (Movie/Booking/Payment).

	8. Gateway returns the response from the target service to the client.
Alternative Flows:	4a. Invalid Token: If User Service returns 401, Gateway denies request immediately.
Exceptions:	2a. Invalid API Key: Returns 401 Unauthorized. 5a. User Service Down: Returns 500 Internal Server Error.
Priority:	High

UC-02: User Registration: Allows new users to create an account.

ID and Name:	UC-02: User Registration
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	Allows a new user to create an account in the system.
Trigger:	Guest sends: <i>POST /api/auth/register</i>
Preconditions:	PRE-1: Email must not already exist in users.db.
Postconditions:	POS-1: New user record created with role 'user'.
CòMIRNormal flow:	1. User submits email and password. 2. Gateway forwards to User Service. 3. User Service hashes the password. 4. User Service saves the record to users.db. 5. System returns success message.
Alternative Flows:	none
Exceptions:	4a. Email Exists: Returns 400 Bad Requests.
Priority:	High

UC-03: User Login: Authenticates users and issues Access Tokens (JWT).

ID and Name:	UC-03: User Login
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User, Admin
Description:	Authenticates a user and issues a <i>Database-backed UUID Token</i> for accessing protected resources.
Trigger:	User sends <i>POST /api/auth/login</i>
Preconditions:	PRE-1: User/admin account must exist.
Postconditions:	POS-1: System returns a valid Access Token. POS-2: Client stores this token for future requests.
Normal flow:	1. User submits email and password. 2. Gateway forwards request to User Service. 3. User Service verifies email exists. 4. User Service validates password hash. 5. User Service generates a random <i>UUID Token</i>, stores it in the <i>tokens</i> table, and returns it. 6. System returns the Token to the user.
SAAlternative Flows:	none
Exceptions:	3a. User Not Found / Wrong Password: Returns 401 Unauthorized.
Priority:	High

5.2 Movie & Showtime Management (Admin) (UC-04 to UC-09)

UC-04: Create Movie: Adds a new movie to the system.

ID and Name:	UC-04: Create Movie
--------------	---------------------

Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Adds a new movie title to the system database.
Trigger:	Admin sends POST /api/movies
Preconditions:	PRE-1: User has 'admin' role. PRE-2: Movie ID must be unique.
Postconditions:	POS-1: New movie record is inserted into movies.db.
Normal flow:	1. Admin submits movie details (ID, Title, Genre, Duration). 2. Gateway checks Admin Token. 3. Movie Service checks if ID already exists. 4. Movie Service inserts data into movies table. 5. System returns "Movie Created" message.
Alternative Flows:	none
Exceptions:	3a. Duplicate ID: Returns 400 Bad Request.
Priority:	High

UC-05: Update Movie: Updates existing movie information.

ID and Name:	UC-05: Update Movie
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Allows Admin to correct or update movie information (Title, Genre, Duration).
Trigger:	Admin sends PUT /api/movies/{id}
Preconditions:	PRE-1: Movie exists. PRE-2: User has 'admin' role.

Postconditions:	POS-1: Movie details updated in movies.db.
Normal flow:	1. Admin submits new details (e.g., fix typo in Title). 2. Gateway verifies Admin Token. 3. Movie Service updates the record in database. 4. System returns success message.
Alternative Flows:	none
Exceptions:	none
Priority:	Medium

UC-06: Cascade Delete Movie: Deletes a movie along with its showtimes and triggers automatic ticket refunds.

ID and Name:	UC-06: Cascade Delete Movie
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Deletes a movie and automatically deletes all associated showtimes and refunds all booked tickets.
Trigger:	Admin sends DELETE /api/movies/{id}
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: The Movie ID must exist.
Postconditions:	POS-1: Movie, Showtimes, and Bookings are deleted. POS-2: REFUND_NOTIFICATION is sent to the notification_events queue.
Normal flow:	1. Admin requests deletion. 2. Movie Service finds all associated showtimes.

	3. Loop: Trigger Refund Process for each showtime. 4. Delete all showtimes. 5. Delete the movie record. 6. Returns the count of refunded tickets.
Alternative Flows:	none
Exceptions:	none
Priority:	Medium

UC-07: Create Showtime: Schedules a movie and automatically generates available seats.

ID and Name:	UC-07: Create Showtime
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Schedule a movie and generate seats.
Trigger:	Admin sends POST /api/showtimes
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: The Movie ID must exist. PRE-3: Time is valid. PRE-4: The total number of seats must not exceed 64.
Postconditions:	POS-1: Showtime saved in movies.db. POS-2: Seats created in booking.db.
Normal flow:	1. Admin sends time and price. 2. Movie Service saves showtime. 3. Movie Service calls Booking Service (/internal/seats) to generate seats.

	4. Returns success.
Alternative Flows:	none
Exceptions:	none
Priority:	High

UC-08: Update Showtime & Auto-Refund: Updates showtime details (triggers refunds if the price changes).

ID and Name:	UC-08: Update Showtime & Auto-Refund
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Update showtime. If price changes, existing bookings are automatically cancelled and refunded.
Trigger:	Admin sends PUT /api/showtimes/{id}
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: The Showtime ID must exist.
Postconditions:	POS-1: Update successful. POS-2: (If price changed) Tickets cancelled, REFUND_NOTIFICATION is sent to the notification_events queue.
Normal flow:	1. Admin changes price. 2. Movie Service detects price change. 3. Calls Booking Service to cancel all tickets. 4. Publishes REFUND_NOTIFICATION to RabbitMQ. 5. Updates new price in DB.

Alternative Flows:	none
Exceptions:	none
Priority:	High

UC-09: Delete Showtime: Deletes a specific showtime and refunds any sold tickets.

ID and Name:	UC-09: Delete Showtime
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Delete a specific showtime and refund sold tickets.
Trigger:	Admin sends DELETE /api/showtimes/{id}
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: The Showtime ID must exist.
Postconditions:	POS-1: Showtime deleted. POS-2: REFUND_NOTIFICATION is snotification_events queue.
Normal flow:	1. Admin requests deletion. 2. Gateway verifies Admin Role. 3. Trigger Refund Process. 4. Delete record in DB. 5. Returns success message.
Alternative Flows:	none
Exceptions:	none
Priority:	High

5.3 Browsing & Booking (User) (UC-10 to UC-15)

UC-10: Search Movies: Lists all movies or searches for specific ones by title/ID.

ID and Name:	UC-10: Search Movies
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User, Admin
Description:	List all movies or search by title or show one movies by id
Trigger:	GET/api/movies(with?query=...) GET/api/movies/{id}
Preconditions:	PRE-1: Movie Service is running. PRE-2: Database has movie records (optional).
Postconditions:	POS-1: Returns list of movies matching query. POS-2: No database modification.
Normal flow:	1. User sends query. 2. Service performs LIKE search. 3. Returns list.
Alternative Flows:	none
Exceptions:	none
Priority:	Medium

UC-11: View Showtimes Schedule: Displays available showtimes and pricing.

ID and Name:	UC-11: View Showtimes Schedule
Create by:	Nguyen Gia Bao / Date created: 18/01/2026

Primary Actor:	User, Admin
Description:	View list of available showtimes and prices.
Trigger:	GET /api/showtimes
Preconditions:	PRE-1: Movie Service is running.
Postconditions:	POS-1: Returns list of showtimes. POS-2: No database modification.
Normal flow:	1. User requests schedule. 2. System returns list of showtimes.
Alternative Flows:	none
Exceptions:	none
Priority:	High

UC-12: Book Ticket: Reserves a seat and publishes an order event to the system.

ID and Name:	UC-12: Book Ticket
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	User reserves a seat. Price is fetched dynamically.
Trigger:	POST /api/bookings
Preconditions:	PRE-1: User must be logged in. PRE-2: Seat must be 'available'.
Postconditions:	POS-1: Booking created. POS-2: The OrderPlacedEvent event is sent to the ticket_events queue.
Normal flow:	1. User selects seat.

	2. Service checks availability. 3. Calls Movie Service to fetch price. 4. Creates booking and updates seat status. 5. Sends event to RabbitMQ.
Alternative Flows:	none
Exceptions:	2a. Seat Taken: Returns 400 Bad Request.
Priority:	High

UC-13: View Bookings: Allows users to view their personal booking history.

ID and Name:	UC-13: View Bookings
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	View history of bookings made by the user.
Trigger:	GET /api/bookings
Preconditions:	PRE-1: User must be logged in.
Postconditions:	POS-1: Returns list of bookings for that user.
Normal flow:	1. Gateway extracts email from Token. 2. Service filters bookings by email. 3. Returns list.
Alternative Flows:	none
Exceptions:	none
Priority:	High

UC-14: View Booking Detail: Shows specific details for a single ticket.

ID and Name:	UC-14: View Booking Detail
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	View specific details of a single ticket.
Trigger:	GET /api/bookings/{id}
Preconditions:	PRE-1: User must be logged in. PRE-2: User must own the ticket.
Postconditions:	POS-1: Returns ticket details.
Normal flow:	1. User requests ID. 2. System verifies ownership. 3. Returns details.
Alternative Flows:	none
Exceptions:	2a. Not Owner: Returns 403 Forbidden.
Priority:	High

UC-15: Cancel Booking (Manual): Allows users to manually cancel their own bookings.

ID and Name:	UC-15: Cancel Booking (Manual)
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	User cancels their own ticket. Note: Manual cancellation does NOT trigger a refund event in the current implementation.
Trigger:	DELETE /api/bookings/{id}
Preconditions:	PRE-1: User must be logged in. PRE-2: User must own the ticket.
Postconditions:	POS-1: Booking deleted.

	POS-2: Seat becomes available.
Normal flow:	1. User requests cancellation. 2. System verifies ownership. 3. Deletes booking. 4. Frees up seat.
Alternative Flows:	none
Exceptions:	none
Priority:	Medium

5.4 Payment (UC-16)

UC-16: Process Payment: Processes mock transactions, logs payments, and generates invoices.

ID and Name:	UC-16: Process Payment
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	Pay for a ticket and generate an invoice.
Trigger:	POST /api/payments
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: Valid Booking ID exists.
Postconditions:	POS-1: Payment logged. POS-2: INVOICE_PRINT event sent.
Normal flow:	1. User submits Booking ID. 2. Gateway verifies Token and routes to Payment Service.

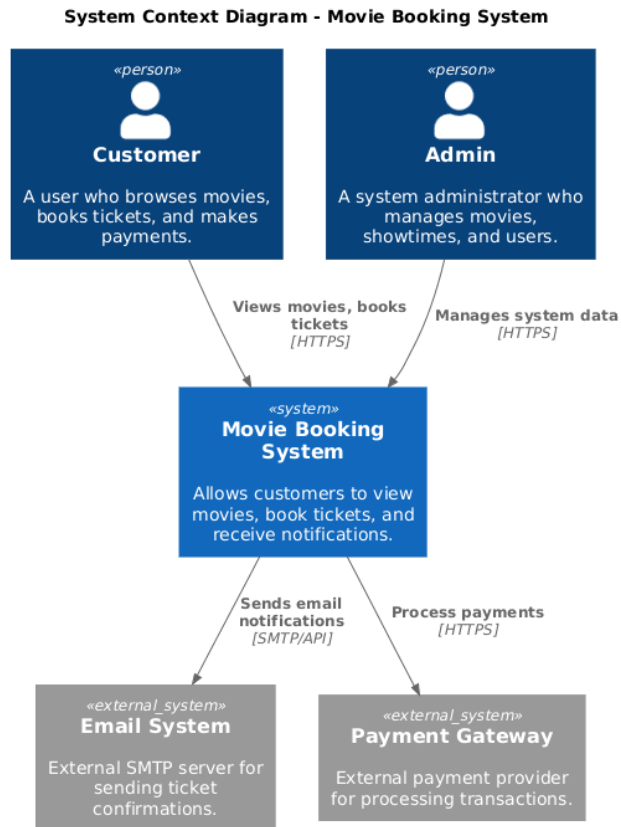
	3. Payment Service calls Booking Service (Internal API) to verify the booking exists and get the correct amount. 4. Payment Service processes the mock transaction. 5. Payment Service publishes InvoiceGenerated event to RabbitMQ. 6. System returns "Payment Successful" message to User.
Alternative Flows:	none
Exceptions:	3a. Booking Not Found: Payment Service returns 404. 3b. Already Paid: Payment Service returns 400 Bad Request.
Priority:	High

6. DETAILED OPERATIONAL MECHANISMS, INTERACTIONS, AND DATA FLOW

The system is designed based on a distributed Microservices architecture, where components do not communicate via internal function calls (in-process calls) but through network calls. This architecture demands strict regulation of communication protocols and data management.

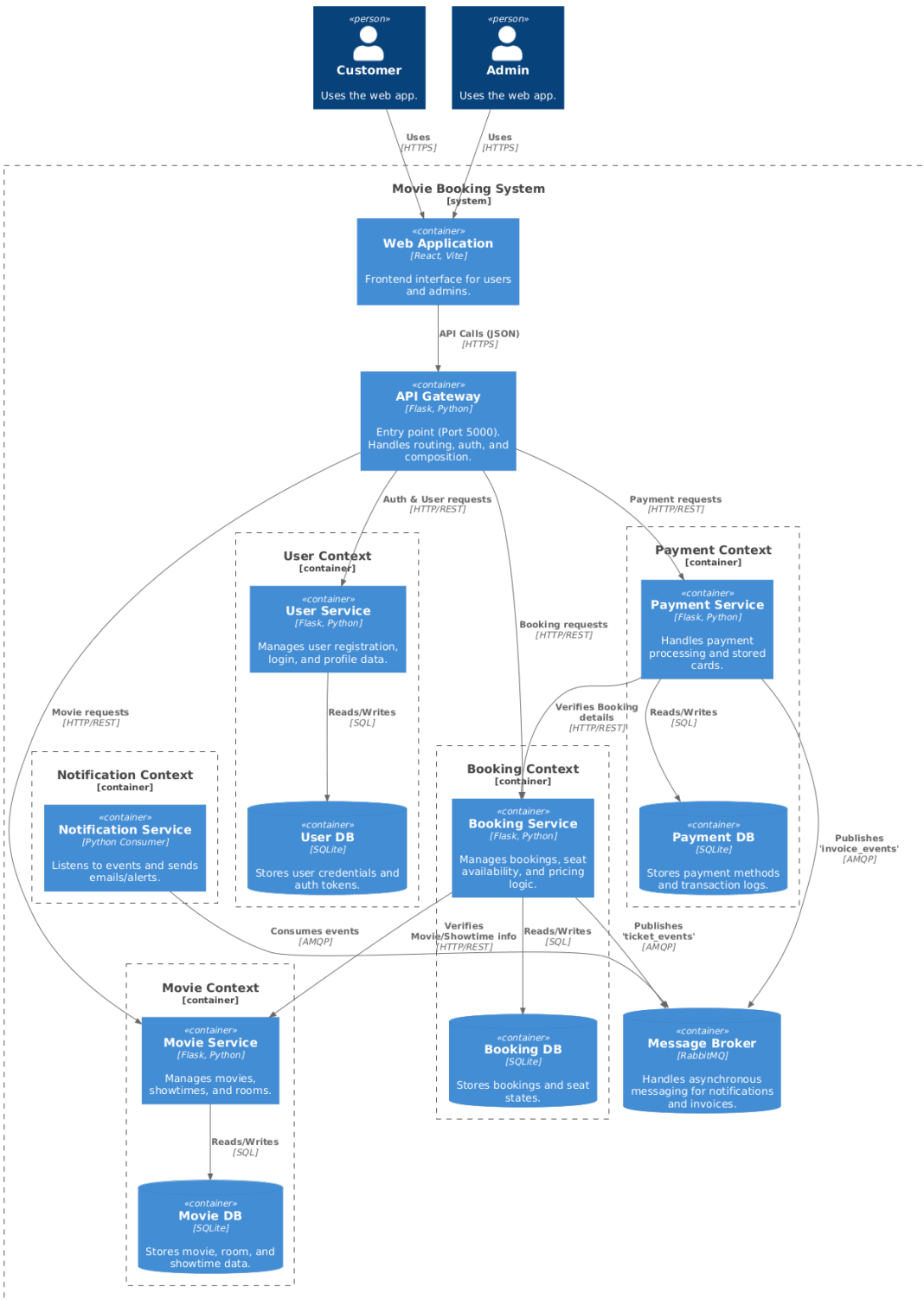
6.1 C4 Model

Level 1: System Context Diagram



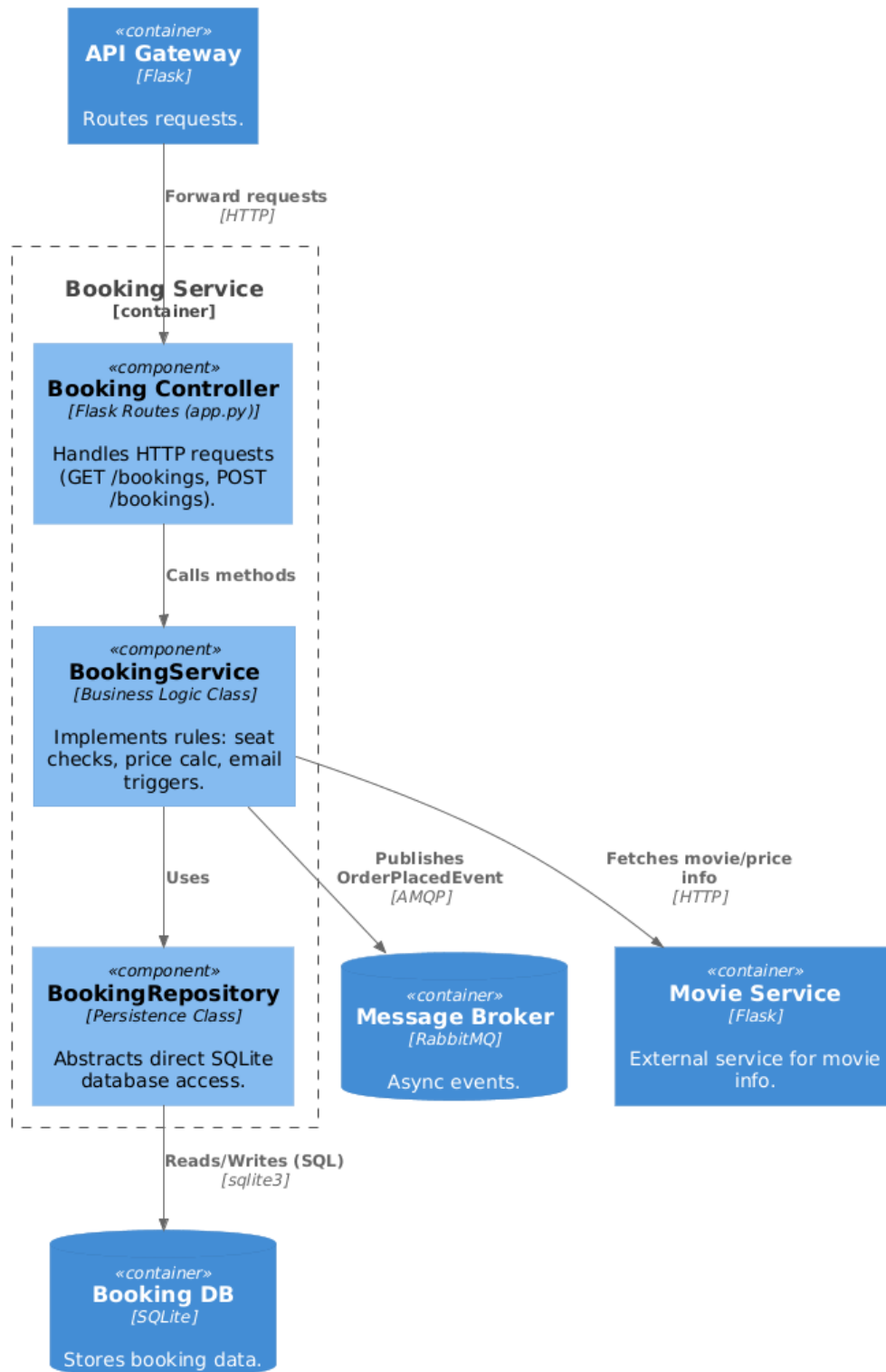
Level 2: Container Diagram

Container Diagram - Movie Booking System



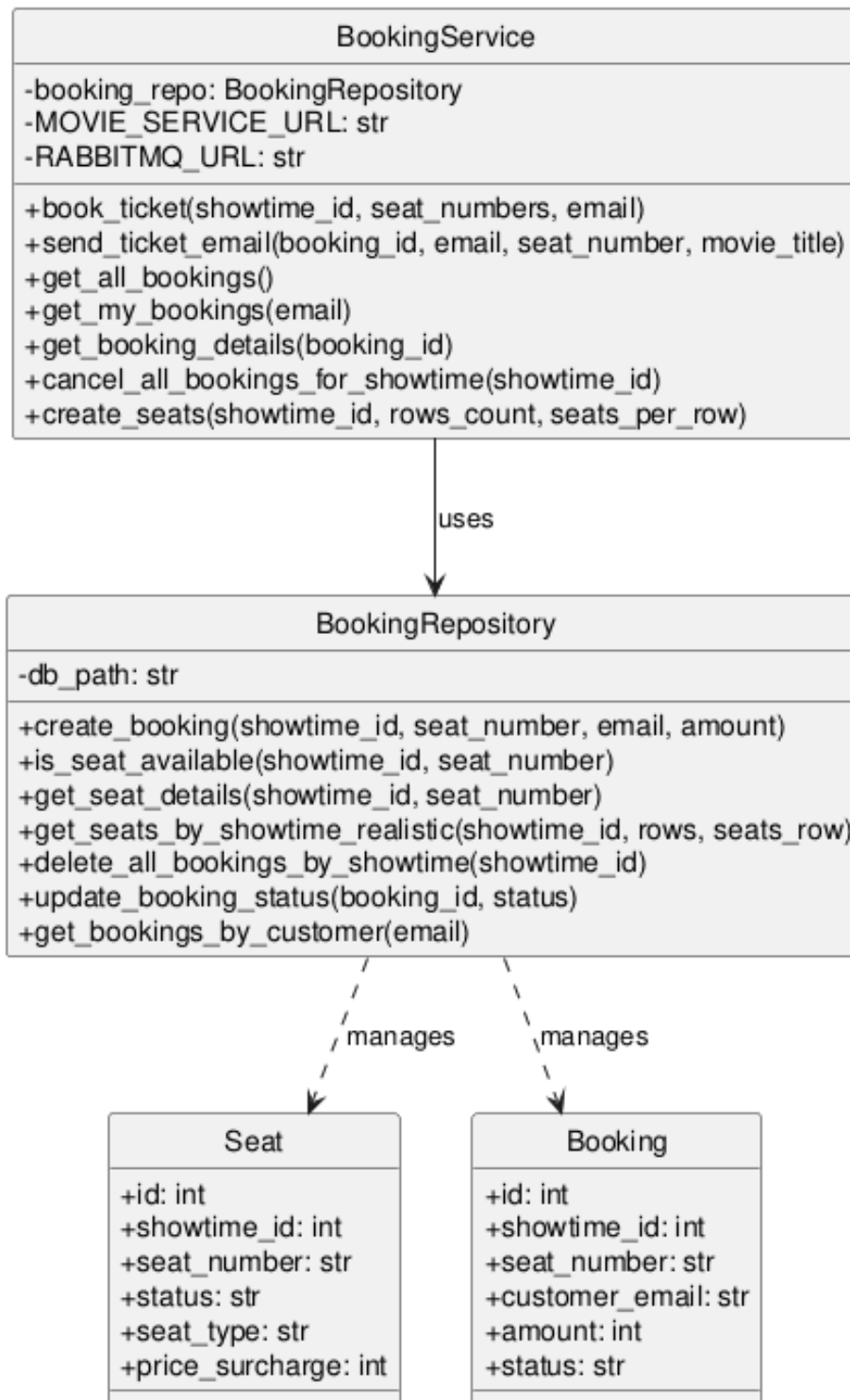
Level 3: Component Level

Component Diagram - Booking Service



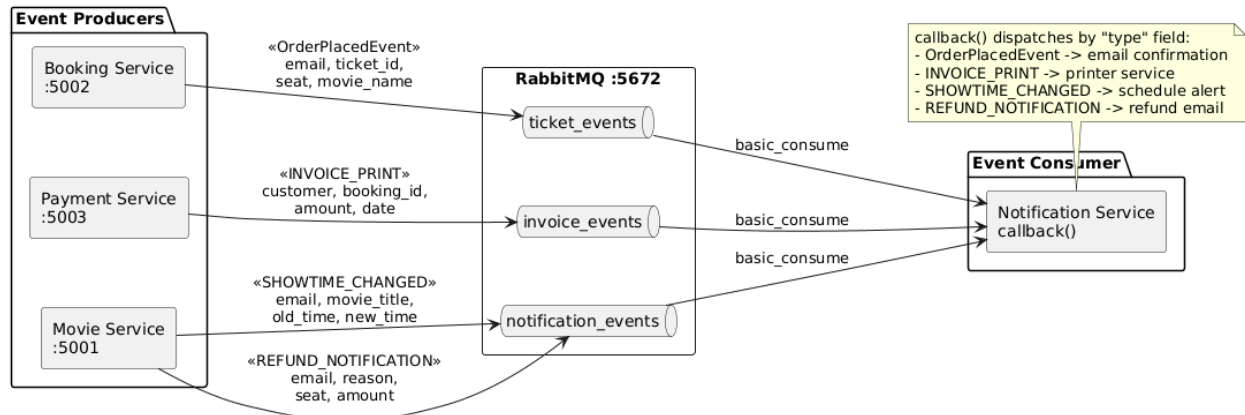
Level 4: Code Diagram

Code Diagram - Booking Service Classes



6.2 Communication Architecture

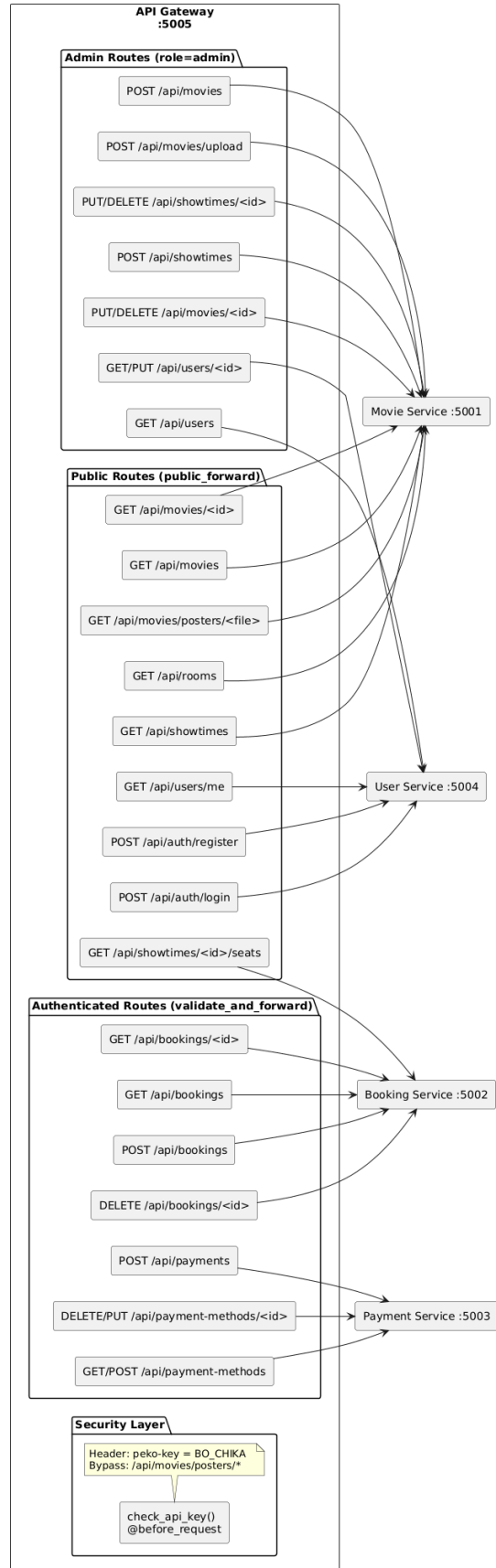
The system employs a Hybrid Communication Pattern to leverage the advantages of both the synchronous and asynchronous worlds:



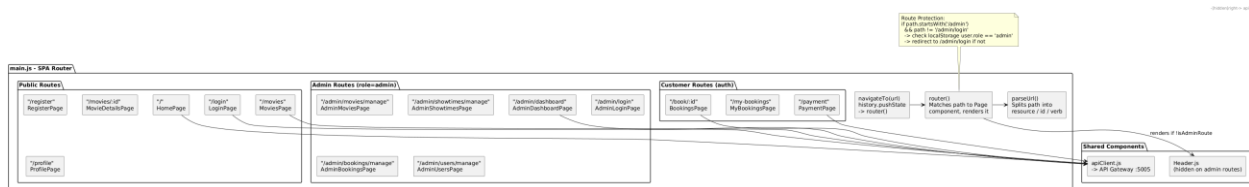
- Synchronous Communication (Request/Response):
 - Protocol: HTTP/1.1 RESTful API.
 - Scope: Tasks requiring immediate feedback to be displayed on the User Interface (UI).
 - Example: When a user accesses the homepage, the Frontend needs to display the movie list immediately. Using an asynchronous mechanism here would cause unnecessary latency and degrade the User Experience (UX).
- Asynchronous Communication (Event-Driven):
 - Protocol: AMQP (Advanced Message Queuing Protocol) via RabbitMQ.
 - Scope: Complex business tasks, time-consuming processing, or side-effects that do not directly affect the main response.
 - Example: Sending ticket emails and generating invoices are consequences of a "Successful Booking". The user does not need to wait for the email to be sent to know that they have finished booking.

6.3 Routing Mechanism and API Gateway Pattern

The API Gateway (running on port 5005) acts as the Single Entry Point for the entire system.



- Reverse Proxy Mechanism: The Gateway intercepts requests from the Client, analyzes the URL Path, and routes them to the corresponding service container within the internal Docker network.
 - Request GET /api/movies -> Forwarded to movie-service:5001.
 - Request POST /api/bookings -> Forwarded to booking-service:5002.



- Architectural Benefits:
 - Abstraction: The Client does not need to know the internal network structure or the number of services behind the scenes.
 - Security: The Gateway acts as a firewall, blocking invalid requests before they reach the business services.

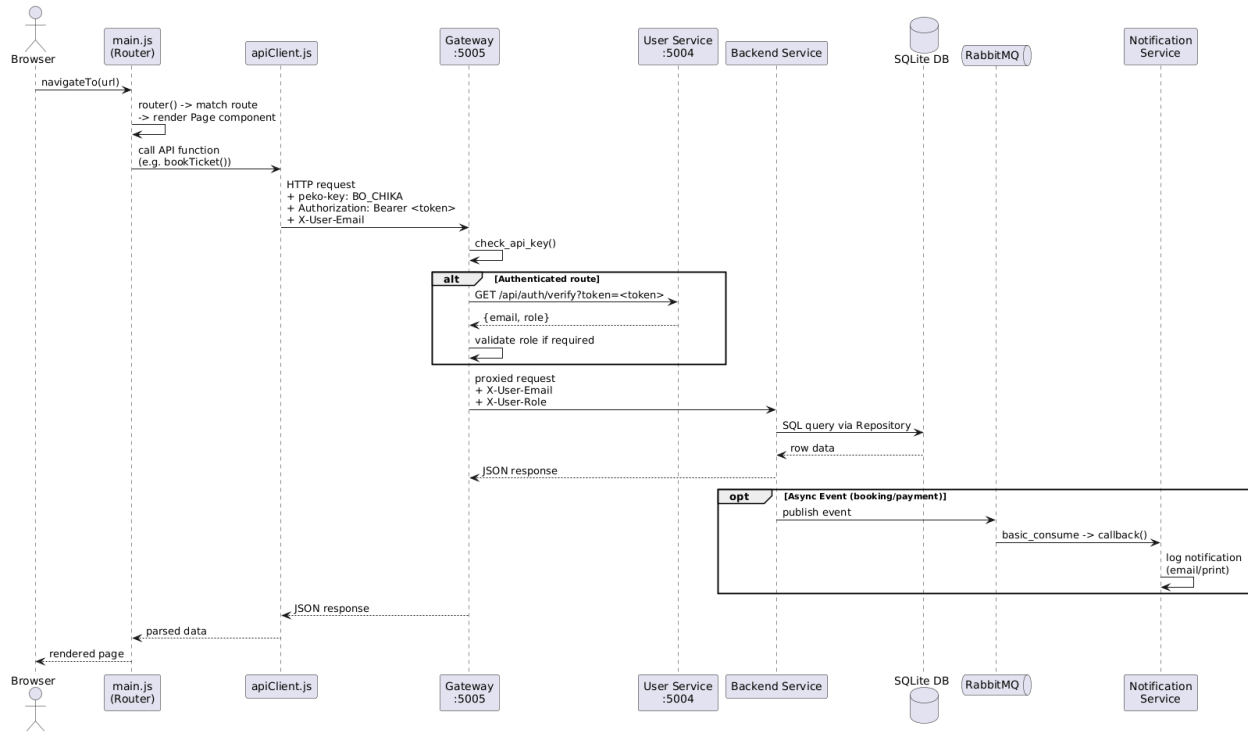
6.4 Detailed Workflows Analysis

A. Data Query Flow (Read-Heavy Path)

This process focuses on Read Throughput.

1. Initiation: The Frontend calls the API via the Gateway.
2. Service Resolution: The Gateway uses Docker DNS to resolve the hostname movie-service to the dynamic IP address of the running container.
3. Data Retrieval: The Movie Service connects to its isolated SQLite database (movies.db). The service executes a SELECT SQL query to retrieve raw data.
4. Serialization: Data from the Database (Rows/Tuples) is serialized into standard JSON format.
5. Response: The JSON is sent back through the Gateway to the Frontend for rendering.

B. Distributed Booking Transaction



This is the most complex process, applying the "Choreography" model in the Saga Pattern (simplified) to ensure data consistency.

- Step 1: Validation & Locking (At Booking Service)
 - The Service receives a seat booking request.
 - Executes "Check-then-Act": Checks booking.db to see if the seat is already taken.
 - Uses a local SQLite Transaction to ensure Atomicity: If the seat is free, it inserts a Booking record with *PENDING_PAYMENT* status and locks the seat, only switch to *CONFIRMED* after the payment is successfully completed.
 - If this step fails (Database error, duplicate seat), the API returns an error immediately (Fail Fast).
- Step 2: Event Publication (At Booking Service)
 - After a successful commit to the Database, the Booking Service acts as a Producer.
 - It creates a Message containing the minimum necessary information (Payload): BookingID, UserID, Amount, ShowtimeInfo.

- The Message is pushed directly to the RabbitMQ Queue (*ticket_events*).
 - *Note:* The response to the user (HTTP 201 Created) occurs in parallel or immediately after pushing the message, ensuring an extremely fast interface response (Non-blocking I/O).
- Step 3: Message Routing (At RabbitMQ)
 - The Booking Service sends messages directly to the *ticket_events* queue. The Payment Service (upon successful transaction) sends messages to the *invoice_events* queue.
- Step 4: Event Consumption (At Notification & Payment Services)
 - These two services operate independently (Decoupled).
 - Notification Service: Consumes messages from multiple queues (**ticket_events** for emails, **invoice_events** for invoices). It uses the payload info to render the HTML ticket template and sends it via the SMTP Server.
 - Payment Service: Exposes a REST API to process transactions. Upon success, it publishes an *INVOICE_PRINT* event to the *invoice_events* queue for the Notification Service to consume.
 - Acknowledge Mechanism (ACK): After processing is complete, the Service sends an ACK signal to RabbitMQ to delete the message from the queue. If the Service crashes during processing, the message is not lost but is re-queued by RabbitMQ for later processing.

6.5 Distributed Data Management

The system adheres to the Database-per-Service principle:

- Independence: The Movie Service cannot directly access booking.db and vice versa. This prevents "Coupling" at the data layer.
- Data Sharing: When the Booking Service needs movie information (e.g., Movie Title, Showtime) to save on the ticket, it does not join tables (since they are in different databases). Instead, the Frontend sends this information in the payload, or the Booking Service stores a reference ID (*movie_id*) and makes a synchronous API call to the Movie Service if verification is needed.

6.6 Fault Tolerance

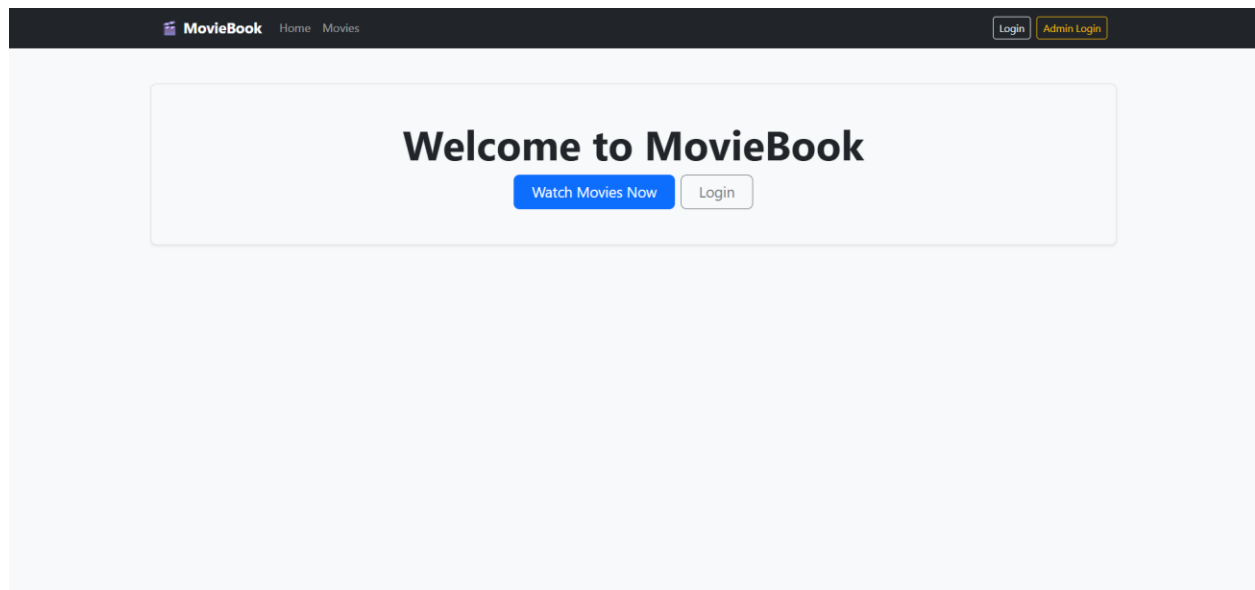
This architecture is designed to prevent a total system collapse when a single component fails (Single Point of Failure elimination - excluding the Gateway):

- Scenario: If the Notification Service crashes.
- Consequence: The user still books the ticket successfully. The email is simply not sent immediately.
- Self-Healing: The message remains safe in the RabbitMQ queue. When the Notification Service is restarted (by Docker Compose), it will consume the backlog of messages and send the emails retroactively. This is a critical "Resilience" feature of the system.

7. IMPLEMENTATION: CLIENT-SIDE LOGIC

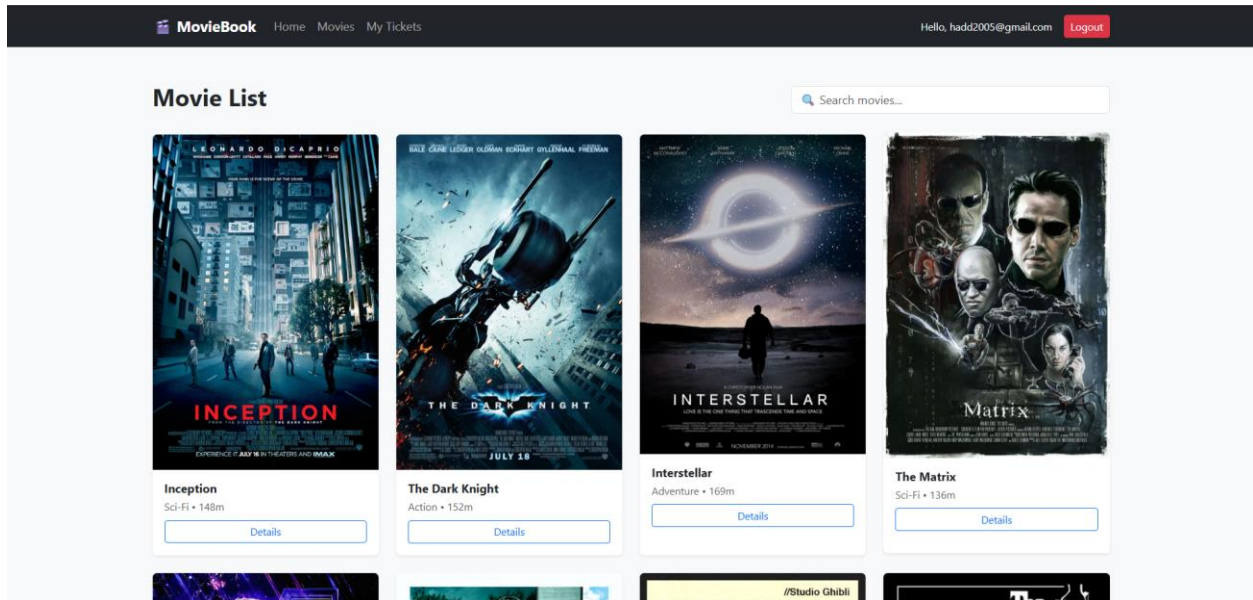
7.1 Customer

Step 1: Browse Movies

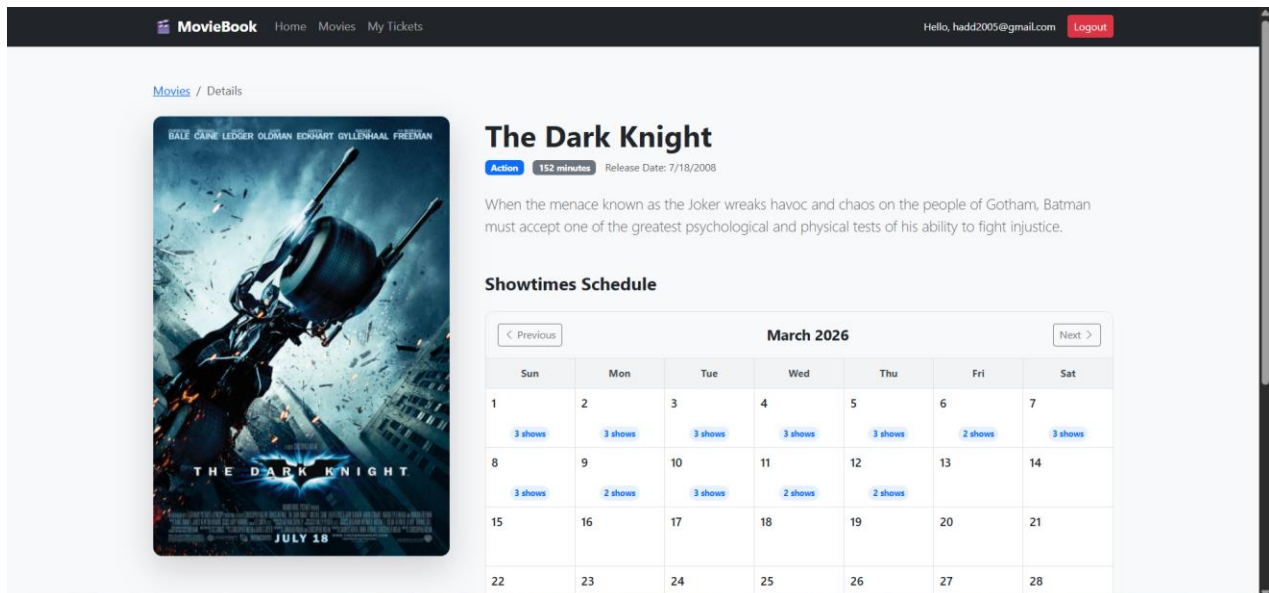


- The customer starts at the **HomePage (/)** and selects “Watch Movies Now” or navigates to /movies.
- **MoviesPage** sends a request GET /api/movies through the **API Gateway** to retrieve the list of movies.
- The request is forwarded to the **Movie Service**.
- Movies are displayed as cards containing the **poster, title, genre, and duration**.
- A **search bar with 500ms debounce** allows users to filter movies.

Step 2: View Movie Details & Pick a Showtime



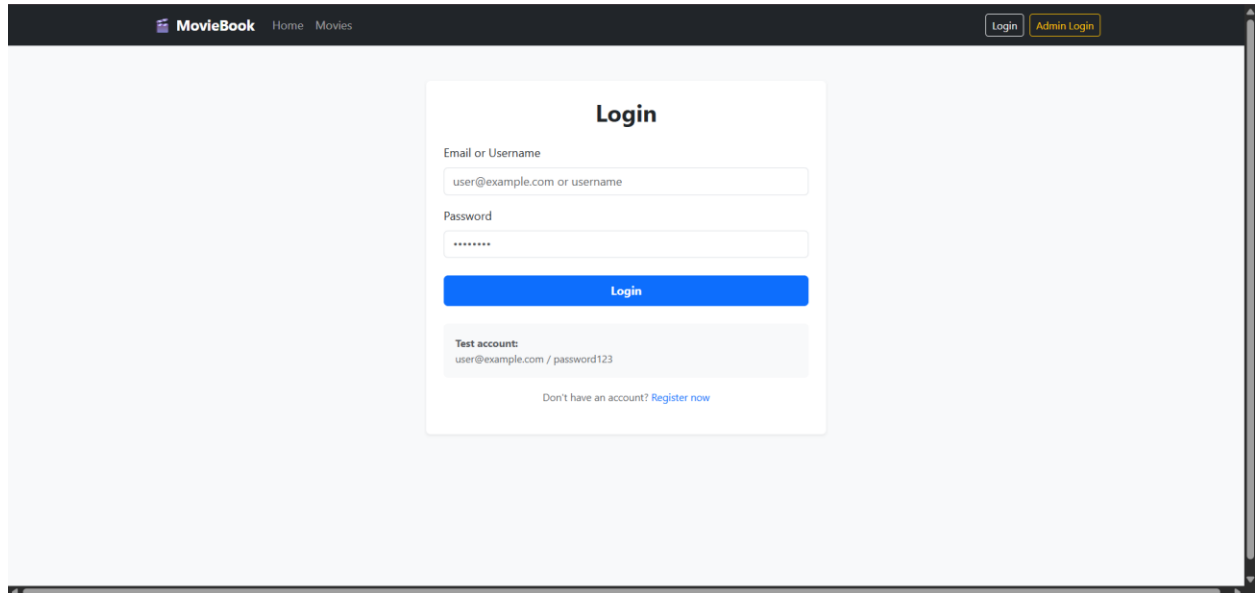
- When the user selects a movie, the system navigates to `/movies/:id`.
- **MovieDetailsPage** makes three parallel API calls:
 - `getMovieById(id)` → movie information
 - `getShowtimes(id)` → list of showtimes
 - `getRooms()` → room information
- Showtimes are grouped by date and displayed in a **calendar view**.



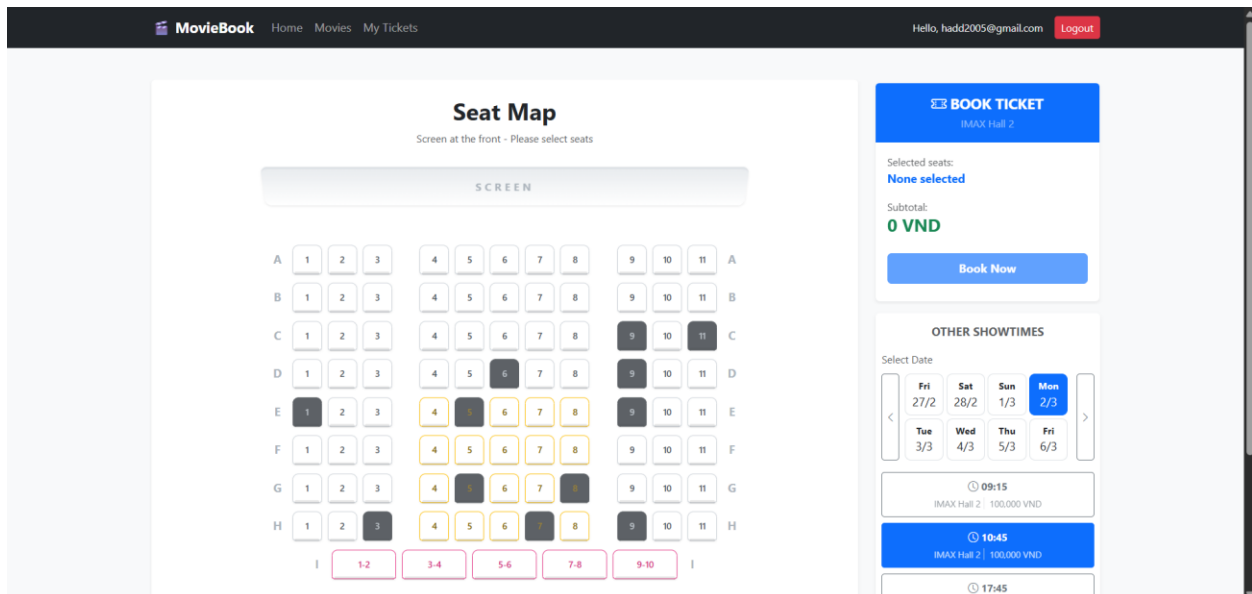
- Each showtime shows the **time, room name, and ticket price**.
- Selecting a showtime redirects the user to `/book/{showtime_id}`.

Step 3: Select Seats

- The **BookingsPage (/book/:id)** is loaded.
- If the user is not logged in, a **login prompt** is displayed.



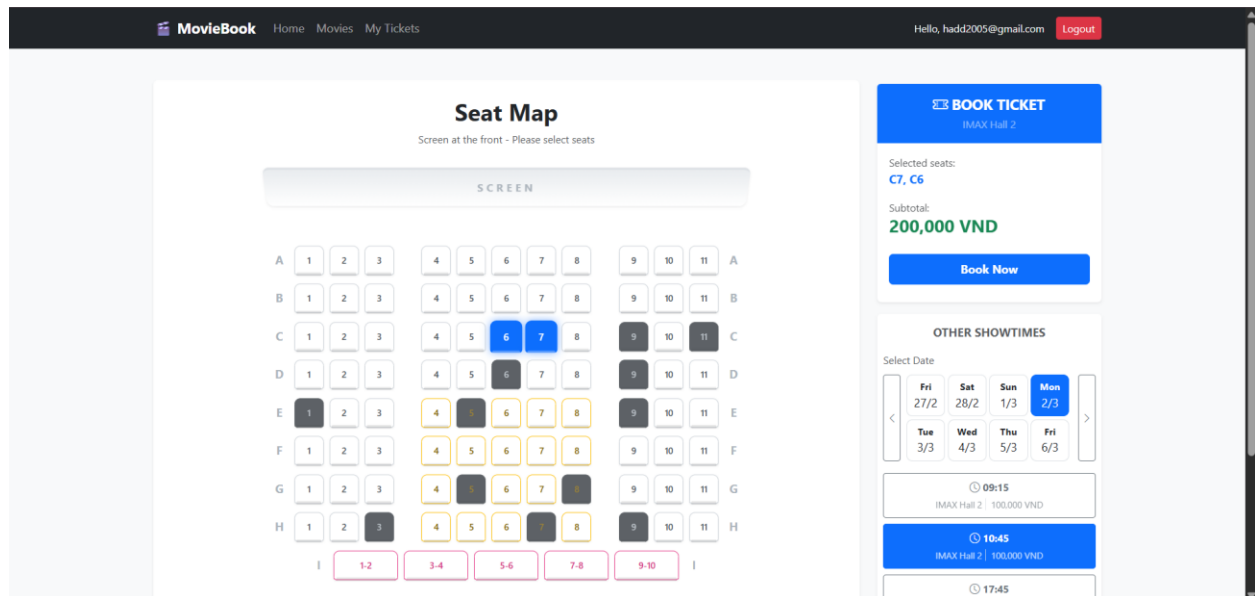
- If logged in:
 - The system retrieves showtime details using `getShowtimeDetail`.
 - The seat map is retrieved via `GET /api/showtimes/:id/seats`.
- The **API Gateway** routes the seat request to the **Booking Service**.



- **Booking Service:**
 - Retrieves the room layout from **Movie Service**.
 - Initializes seat data if it does not exist.

- Seats are displayed in rows (A–I).
- Seat types include:
 - **STANDARD**
 - **VIP** (+20k)
 - **COUPLE** (+50k)
- Users can select or deselect seats.
- The sidebar updates the **selected seats and running subtotal** in real time.

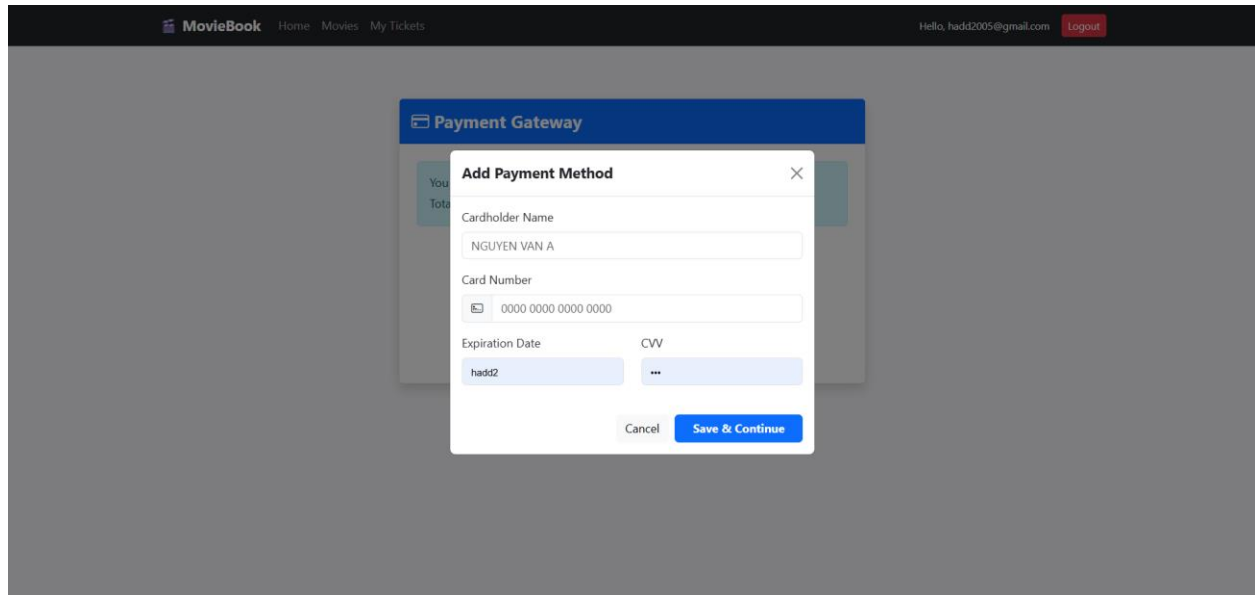
Step 4: Confirm Booking



- When the user clicks **Book Now**, the frontend sends:
 - POST /api/bookings
 - { showtime_id, seat_numbers }
- The **API Gateway**:
 - verifies the **Bearer token**
 - retrieves the user email from the **User Service**
 - forwards the request to the **Booking Service**.
- **BookingService.book_ticket()** performs:
 - seat availability validation
 - retrieval of ticket price from **Movie Service**
 - total price calculation
 - creation of a booking record for all seats (atomic transaction)
 - publishing an OrderPlacedEvent to **RabbitMQ**.

Step 5: Payment

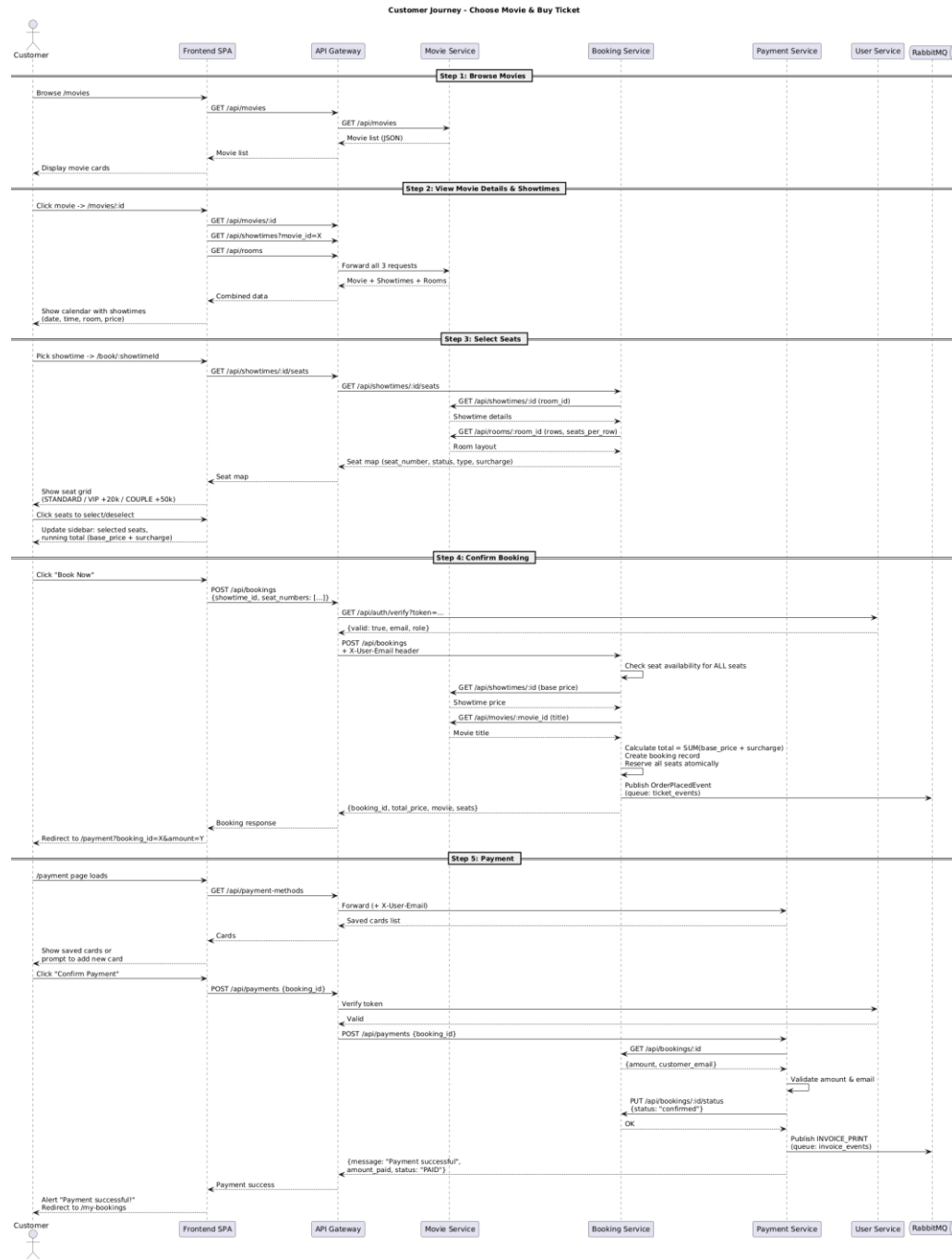
- After the booking is created, the user is redirected to /payment.



The screenshot displays the 'MovieBook' application interface. At the top, a dark navigation bar contains the 'MovieBook' logo, links for 'Home', 'Movies', and 'My Tickets', and a user profile section showing 'Hello, hadd2005@gmail.com' and a 'Logout' button. The main content area is a light gray. A 'Payment Gateway' modal is open, featuring a blue header with a close button. The modal contains the following fields: 'Cardholder Name' with the value 'NGUYEN VAN A', 'Card Number' with a masked value '0000 0000 0000 0000', 'Expiration Date' with the value 'hadd2', and 'CVV' with a masked value '***'. At the bottom of the modal are 'Cancel' and 'Save & Continue' buttons.

- **PaymentPage:**
 - loads saved payment cards using `getPaymentMethods()`.
 - the user selects a card or adds a new one.
- When confirming payment:
 - `POST /api/payments` is called.
- **Payment Service:**
 - retrieves booking details from **Booking Service**
 - verifies the payment amount and user email
 - updates the booking status to **confirmed**
 - publishes an **INVOICE_PRINT** event to **RabbitMQ**.
- Finally, the user is redirected to **/my-bookings** to view purchased tickets.

Summary Flow Diagram:



7.2 Admin

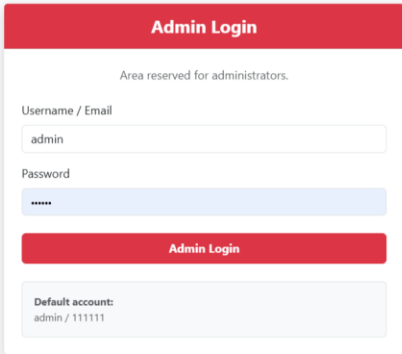
7.2.1 Introduction

The **MovieBook** system provides a dedicated **Admin Module** for managing core entities:

- Movies
 - Showtimes
 - Bookings (tickets)
 - User accounts
- All admin operations are protected by **role-based access control (RBAC)** implemented at two layers:
 - **Client-side** route guards
 - **Server-side** API Gateway token and role verification
- The admin interface is available under the route **/admin/***.
- The admin panel includes a sidebar with navigation links to:
 - Dashboard
 - Manage Movies
 - Manage Showtimes
 - Manage Bookings
 - Manage Users

7.2.2 Authentication and Authorization

– Admin Login



Admin Login

Area reserved for administrators.

Username / Email

admin

Password

Admin Login

Default account:
admin / 111111

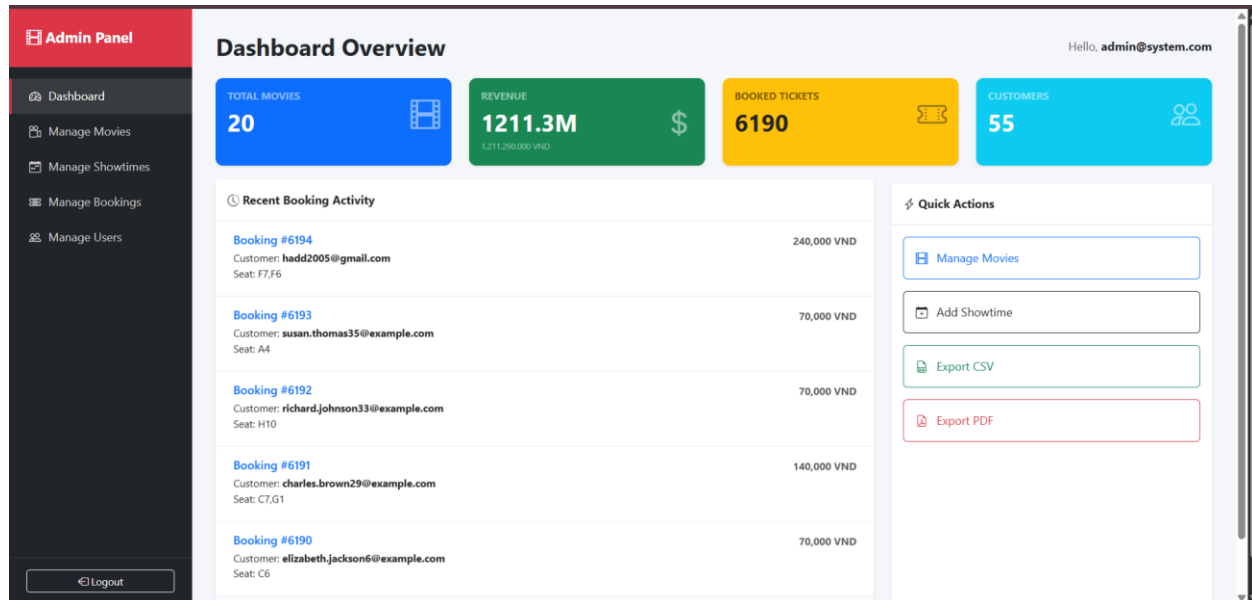
← Back to Home

- The system initializes a **default admin account** on startup:

- Email: admin@system.com
 - Password: 111111
- Admin login is handled at **/admin/login**, which is separate from the customer login page.
- After login:
 - The frontend verifies that **response.role === 'admin'**.
 - If the role is not admin, the login is rejected with the message:
 - *“This account does not have admin privileges.”*
- On successful login:
 - The **user object**, including **token and role**, is stored in **localStorage**.
- **Client-Side Route Protection**
- All **/admin/* routes** (except /admin/login) are protected by the frontend router.
- Before rendering a page:
 - The router checks **localStorage** for the logged-in user.
 - It verifies **user.role === 'admin'**.
- If the check fails:
 - The user is redirected to **/admin/login**.
- **Server-Side Role Enforcement**
- Backend authorization is handled by the **API Gateway** via the **validate_and_forward()** function.
- For **admin-only endpoints**, the gateway specifies:
 - **required_role = 'admin'**.
- The authorization process includes:
 - Extracting the **Bearer token** from the Authorization header.
 - Verifying the token using **GET /api/auth/verify** on the **User Service**.
 - Comparing the returned role with the **required role**.
- If the role does not match:
 - The system returns **HTTP 403 Forbidden**.
- If valid:
 - The gateway forwards the request and injects headers:
 - X-User-Email
 - X-User-Role

7.2.3 Admin Dashboard

Route: /admin/dashboard



– Overview Statistics

- The dashboard loads statistics from multiple services in parallel using **Promise.all()**:
 - `getMovies()` → total movie count
 - `getAllBookings()` → total bookings and revenue calculation
 - `getAllUsers()` → total customers (filtered by role === 'customer')
 - `getShowtimes()` → showtime data for cross-referencing
- The dashboard displays **four statistics cards**:
 - Total Movies
 - Total Revenue
 - Booked Tickets
 - Customers
- It also shows **five most recent bookings** (sorted by ID descending) under **Recent Activity**.

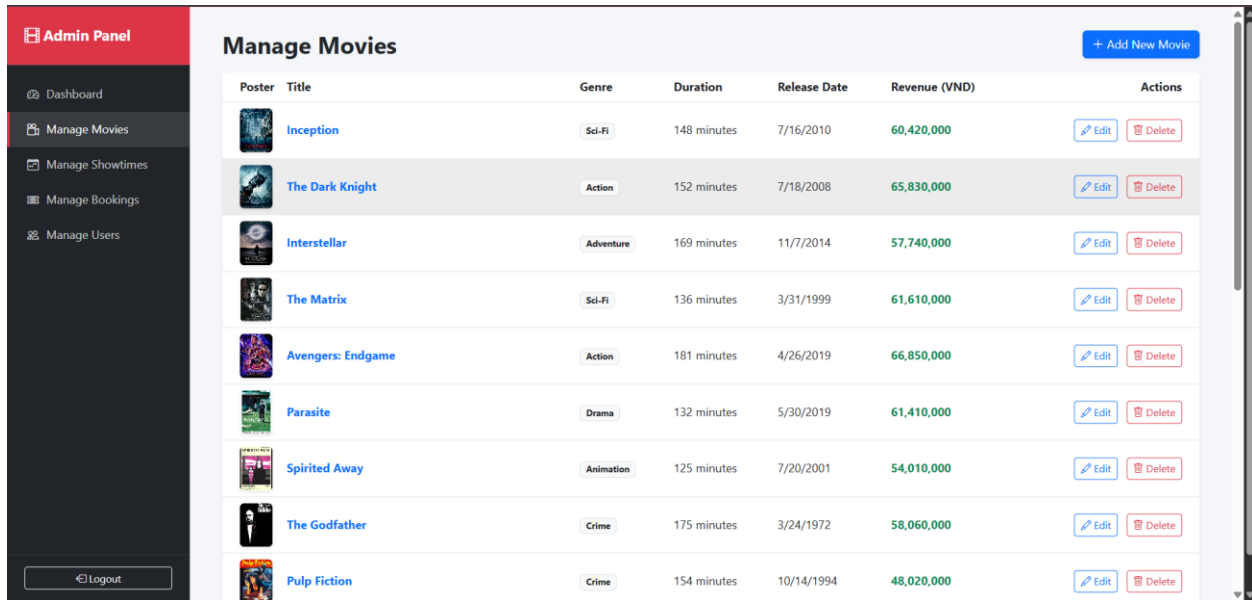
– CSV Report Export

- CSV reports are generated entirely **on the client side**.
- The system cross-references:
 - movies
 - showtimes

- bookings
- Revenue is calculated **per movie**.
- CSV columns include:
 - Movie ID
 - Title
 - Genre
 - Release Date
 - Total Bookings
 - Total Revenue (VND)
- File downloaded as:
 - revenue_report_YYYY-MM-DD.csv.
- **PDF Report Export**
- PDF reports are generated using:
 - **jsPDF**
 - **jspdf-autotable plugin**
- The report contains:
 - Title
 - Report generation date
 - Table of revenue per movie
 - Total system revenue
- File downloaded as:
 - revenue_report_YYYY-MM-DD.pdf.

7.2.4 Movie Management

Route: /admin/movies/manage



Poster	Title	Genre	Duration	Release Date	Revenue (VND)	Actions
	Inception	Sci-Fi	148 minutes	7/16/2010	60,420,000	Edit Delete
	The Dark Knight	Action	152 minutes	7/18/2008	65,830,000	Edit Delete
	Interstellar	Adventure	169 minutes	11/7/2014	57,740,000	Edit Delete
	The Matrix	Sci-Fi	136 minutes	3/31/1999	61,610,000	Edit Delete
	Avengers: Endgame	Action	181 minutes	4/26/2019	66,850,000	Edit Delete
	Parasite	Drama	132 minutes	5/30/2019	61,410,000	Edit Delete
	Spirited Away	Animation	125 minutes	7/20/2001	54,010,000	Edit Delete
	The Godfather	Crime	175 minutes	3/24/1972	58,060,000	Edit Delete
	Pulp Fiction	Crime	154 minutes	10/14/1994	48,020,000	Edit Delete

– Listing Movies

- All movies are loaded via:
 - **GET /api/movies**
- Each row displays:
 - Poster thumbnail
 - Title
 - Genre
 - Duration
 - Release date
 - Showtime count
 - Revenue
- Revenue is calculated by cross-referencing:
 - bookings
 - showtimes
- A **search bar** allows filtering by **movie title**.

– Creating and Updating Movies

- Movies are created or edited using a **modal form** with fields:
 - Title
 - Genre

- Duration
 - Release date
 - Poster (file or URL)
 - Description
- If a **poster file** is uploaded:
 - It is sent to **POST /api/movies/upload**.
 - The file is stored in the Movie Service **/static/ directory**.
 - The returned filename is used as **image_url**.
- Form submission endpoints:
 - **Create:** POST /api/movies
 - **Update:** PUT /api/movies/:id
- **Deleting Movies (Cascading)**
- MovieService.delete_movie() performs a **cascade operation**:
 - Retrieves all showtimes related to the movie.
 - For each showtime:
 - Calls process_refund_and_cancel().
 - Retrieves affected bookings from **Booking Service**.
 - Publishes **REFUND_NOTIFICATION** events to **RabbitMQ**.
 - Cancels bookings via DELETE /api/bookings/:id.
 - Deletes all related showtimes.
 - Deletes the movie record.
- Response example:
 - *“Movie deleted. Removed X showtimes. Refunded Y bookings.”*
- **Gateway Routing**
- GET /api/movies → public access
- POST /api/movies → requires **admin role**
- PUT /api/movies/:id → requires **admin role**
- DELETE /api/movies/:id → requires **admin role**

7.2.5 Showtime Management

Route: /admin/showtimes/manage

Admin Panel - Dashboard - Manage Movies - Manage Showtimes - Manage Bookings - Manage Users Logout	Manage Showtimes					03/05/2026	+ Add Showtime
	Movie	Start	End	Ticket Price (VND)	Revenue (VND)	Actions	
	Avengers: Endgame IMAX Hall 2	09:00	12:01	100,000	1,490,000	Edit	Delete
	Seven Samurai Standard Hall 1	09:30	12:57	50,000	950,000	Edit	Delete
	Interstellar IMAX Hall 2	10:15	13:04	100,000	1,280,000	Edit	Delete
	Forrest Gump IMAX Hall 1	10:15	12:37	100,000	0	Edit	Delete
	Parasite IMAX Hall 1	12:00	14:12	100,000	0	Edit	Delete
	The Godfather Standard Hall 2	12:00	14:55	50,000	490,000	Edit	Delete
	Seven Samurai IMAX Hall 1	12:00	15:27	100,000	640,000	Edit	Delete
	Fight Club IMAX Hall 3	12:30	14:49	100,000	1,810,000	Edit	Delete
	Goodfellas IMAX Hall 2	12:30	14:56	100,000	0	Edit	Delete
	The Silence of the Lambs IMAX Hall 2	12:45	14:43	100,000	2,050,000	Edit	Delete
	Inception					Edit	Delete

– Listing Showtimes

- The page loads data in parallel:
 - showtimes
 - movies
 - rooms
 - bookings
- Table columns include:
 - Movie title
 - Room name
 - Date
 - Start and end time
 - Ticket price
 - Booking count
- A **date filter** allows viewing showtimes for a specific day.

– Creating and Updating Showtimes

- Showtimes are managed via a **modal form** with fields:
 - Movie (dropdown)
 - Room (dropdown)
 - Date
 - Start time
 - Ticket price

- End time is calculated automatically:
 - $\text{endDate} = \text{startDate} + \text{movie.duration}$
- Submission endpoints:
 - **Create:** POST /api/showtimes
 - **Update:** PUT /api/showtimes/:id
- Backend validation ensures:
 - $\text{showtime duration} \geq \text{movie duration}$.
- **Updating Showtimes — Impact Analysis**
- **Scenario A: Price or Room Changed**
 - All bookings for the showtime are cancelled.
 - Refund notifications are sent.
 - Reason:
 - Price change affects payment amount.
 - Room change invalidates seat assignments.
 - Response:
 - *“Updated. X bookings refunded due to changes.”*
- **Scenario B: Time Changed**
 - Customers are retrieved from **Booking Service**.
 - **SHOWTIME_CHANGED** events are published to **RabbitMQ**.
 - Each event includes:
 - customer email
 - movie title
 - old time
 - new time
 - Response:
 - *“Showtime updated.”*
- **Scenario C: No Significant Change**
 - Showtime is updated normally.
 - No additional side effects.
- **Deleting Showtimes**
- Deleting a showtime triggers:
 - `process_refund_and_cancel()`
- All related bookings are:

- cancelled
- refunded
- Response example:
 - “Deleted. X bookings refunded.”

7.2.6 Booking (Ticket) Management

Route: /admin/bookings/manage

Admin Panel

Dashboard

Manage Movies

Manage Showtimes

Manage Bookings

Manage Users

Logout

Manage Bookings

Search by email...

ID	User Email	Showtime	Seats	Amount	Status	Actions
#6194	hadd2005@gmail.com	ID: 166	F7,F6	240,000 VND	PENDING_PAYMENT	Cancel
#6193	susan.thomas35@example.com	ID: 1904	A4	70,000 VND	PENDING_PAYMENT	Cancel
#6192	richard.johnson33@example.com	ID: 1904	H10	70,000 VND	confirmed	Cancel
#6191	charles.brown29@example.com	ID: 1904	C7,G1	140,000 VND	confirmed	Cancel
#6190	elizabeth.jackson6@example.com	ID: 1904	C6	70,000 VND	confirmed	Cancel
#6189	robert.martin36@example.com	ID: 1904	A1	70,000 VND	confirmed	Cancel
#6188	elizabeth.johnson4@example.com	ID: 1904	F4	90,000 VND	confirmed	Cancel
#6187	charles.brown29@example.com	ID: 1903	G1,H5	120,000 VND	confirmed	Cancel
#6186	patricia.williams2@example.com	ID: 1903	C11	50,000 VND	confirmed	Cancel
#6185	charles.brown29@example.com	ID: 1903	D11,C5,H8	170,000 VND	confirmed	Cancel
#6184	charles.brown29@example.com	ID: 1903	H2,E9,D5,D7	200,000 VND	confirmed	Cancel
#6183	william.wilson19@example.com	ID: 1902	D1,C6	140,000 VND	PENDING_PAYMENT	Cancel
#6182	thomas.jones39@example.com	ID: 1902	H8,I9-10,E7	300,000 VND	confirmed	Cancel
#6181	patricia.williams2@example.com	ID: 1901	I1-2,E7,H10,G6	290,000 VND	confirmed	Cancel

- Viewing All Bookings
- All bookings are retrieved via:
 - GET /api/bookings
- Table columns:
 - Booking ID
 - Customer email
 - Showtime ID
 - Seat numbers
 - Amount (VND)
 - Status
- Status badges:
 - confirmed (green)
 - pending (yellow)
- Bookings are sorted by:
 - ID descending (most recent first)

- A search bar allows filtering by:
 - customer email
 - booking ID.
- **Cancelling Bookings**
- Admin clicks **Cancel**.
- A confirmation dialog appears.
- If confirmed:
 - The system calls **DELETE /api/bookings/:id**.
- The Booking Service:
 - deletes the booking
 - frees the reserved seats.
- **Gateway Routing**
- GET /api/bookings → requires authentication
- DELETE /api/bookings/:id → requires authentication
- Admin-only access is enforced at the **frontend route guard level**.

7.2.7 User Account Management

Route: /admin/users/manage

ID	Email	Role	Status	Actions
#1	admin@system.com	ADMIN	Active	Edit
#2	test_customer_TestGateway@example.com	CUSTOMER	Active	Edit
#3	user_15aca766@test.com	CUSTOMER	Active	Edit
#4	duplicate_TestErrors@example.com	CUSTOMER	Active	Edit
#5	hadd2005@gmail.com	CUSTOMER	Active	Edit
#6	user@example.com	CUSTOMER	Active	Edit
#7	michael.garcia0@example.com	CUSTOMER	Active	Edit
#8	david.jackson1@example.com	CUSTOMER	Active	Edit
#9	patricia.williams2@example.com	CUSTOMER	Active	Edit
#10	mary.thomas3@example.com	CUSTOMER	Active	Edit
#11	elizabeth.johnson4@example.com	CUSTOMER	Active	Edit
#12	jennifer.martinez5@example.com	CUSTOMER	Active	Edit
#13	elizabeth.jackson6@example.com	CUSTOMER	Active	Edit
#14	patricia.hernandez7@example.com	CUSTOMER	Active	Edit

- **Viewing All Users**
- Users are retrieved via:
 - **GET /api/users**
- This endpoint requires **admin role verification at the API Gateway**.

- Table columns include:
 - User ID
 - Email
 - Role
 - Status badge
- A search bar allows filtering by **email**.
- **Updating User Role and Password**
- Admin opens an **edit modal** for a selected user.
- Editable fields:
 - Role (dropdown)
 - customer
 - admin
 - Password (optional)
- If a password is provided:
 - It is hashed using **generate_password_hash()**.
- Request sent to:
 - **PUT /api/users/:id**
 - Payload: { role, password }.
- **Gateway Routing**
- GET /api/users → requires **admin role**
- PUT /api/users/:id → requires **admin role**

7.2.8 Summary of Admin Operations

- **Movie Management**
- List movies → GET /api/movies
- Create movie → POST /api/movies
- Update movie → PUT /api/movies/:id
- Upload poster → POST /api/movies/upload
- Delete movie → DELETE /api/movies/:id
- Side effects when deleting:
 - cancel bookings
 - send refund notifications
 - delete showtimes

- **Showtime Management**
 - List showtimes → GET /api/showtimes
 - Create showtime → POST /api/showtimes
 - Update showtime → PUT /api/showtimes/:id
 - Delete showtime → DELETE /api/showtimes/:id
 - Side effects:
 - conditional refunds
 - customer notifications
- **Booking Management**
 - View bookings → GET /api/bookings
 - Cancel booking → DELETE /api/bookings/:id
 - Effect:
 - reserved seats become available again.
- **User Management**
 - View users → GET /api/users
 - Update user role → PUT /api/users/:id
 - Reset password → PUT /api/users/:id
- **Dashboard and Reporting**
 - View statistics → multiple service API calls
 - Export revenue report:
 - CSV → client-side file generation
 - PDF → generated using **jsPDF** and **AutoTable**.

8. TESTING & VERIFICATION

8.1 Verification of Requirements (Labs Progress)

The project followed a structured development roadmap, with functionality verified incrementally through an automated integration test suite (*tests/test_api.py*). All 33 test cases were executed through the API Gateway (port 5005), which served as the single entry point to simulate real client behavior.

Functional Verification

All CRUD operations and the complete Booking → Payment → Email workflow were validated using automated integration tests executed through the API Gateway.

- The test runner performs the following steps:
- Starts all 6 microservices as subprocesses
- Waits for health checks (TCP port connectivity with a 45-second timeout)
- Executes tests in logical order:
 - Gateway Security
 - User Service
 - Movie Service
 - Booking Service
 - Payment Service
 - Error Handling

Architecture Verification

The system was successfully decomposed into 8 independent Docker containers:

- gateway
- movie_service
- booking_service
- payment_service
- user_service
- notification_service
- rabbitmq
- frontend

Each service runs independently with its own SQLite database. Communication occurs exclusively through:

- HTTP (synchronous communication)
- RabbitMQ (asynchronous communication)

This confirms proper microservices boundary separation and loose coupling between components.

Security Verification

The Gateway security layer was validated through 5 dedicated test cases (TestGateway), covering:

- Missing API key returns HTTP 401
- Poster endpoints bypass API key requirement
- Admin routes without JWT return HTTP 401
- Customer tokens accessing admin routes return HTTP 403 (RBAC enforcement)
- X-User-Email and X-User-Role headers are correctly forwarded to downstream services

8.2 Quality Attribute Testing

Fault Tolerance Test

We simulated a crash of the Notification Service during ticket booking.

Result:

- Booking completed successfully (HTTP 201 returned to the user)
- RabbitMQ retained the OrderPlacedEvent in the durable ticket_events queue
- After restarting the Notification Service, the email was automatically delivered

This confirms asynchronous decoupling and message durability.

Atomicity Test (Data Consistency)

We tested booking atomicity by attempting to book two seats [seat1, seat2] where seat1 was already reserved.

Result:

- System returned HTTP 400 with "not available" error
- No seats were booked
- No partial transaction occurred

This confirms transactional atomic behavior in the Booking Service.

Cascade Release Test

We validated grouped booking deletion (multiple seats under one booking ID).

Result:

- After DELETE /api/bookings/{id}
- All associated seats returned to status "available"

This confirms correct cascade release logic within seat management.

Retry & Resilience Test

The test infrastructure includes automatic retry logic *within _req()* (TestBase):

- Maximum 3 attempts per request
- 2-second backoff
- Handles transient failures (e.g., SQLite "database is locked" errors)

Result: All 33 test cases passed consistently across multiple Docker runs, demonstrating system stability under sequential load.

Message Broker Integration Test

We published a mock OrderPlacedEvent directly to the ticket_events RabbitMQ queue.

Result:

- Message successfully published
- Notification Service consumed the message via callback()
- Ticket confirmation email dispatched correctly

8.3 Test Summary

Test Suite	Class	Tests	Coverage Area
Gateway Security	TestGateway	5	API key, JWT, RBAC, header forwarding
User Service	TestUser	4	Register, login, profile, admin list
Movie Service	TestMovie	7	CRUD, search, rooms, showtimes
Booking Service	TestBooking	6	Grouped seats, atomicity, cascade release
Payment Service	TestPayment	5	Process payment, card CRUD, 404 handling
Notification	TestNotification	1	RabbitMQ event publishing/consumption
Error Handling	TestErrors	5	404, 400, 401, duplicate registration
Total	7 classes	33	End-to-end via API Gateway

9. CONCLUSION & REFLECTION

9.1 Summary of Lessons Learned

- **Decoupling Complexity:** Moving from Monolith to Microservices significantly increased infrastructure complexity (networking, consistency) but provided superior modularity.
- **Eventual Consistency:** We learned to handle data consistency not through global transactions (ACID), but through Eventual Consistency using Message Queues.

9.2 Future Improvements

- **Caching Layer:** Implement Redis to cache movie schedules, further reducing load on the Movie Service database.
- **CI/CD Pipeline:** Integrate GitHub Actions to automatically run unit tests and build Docker images upon code commit.
- **Centralized Logging:** Implement the ELK Stack (Elasticsearch, Logstash, Kibana) to aggregate logs from all containers for easier debugging.